

Module 3 - Database Queries Assignment

[Start Assignment](#)

- Due Sep 20 by 11:59pm
- Points 100
- Submitting a text entry box or a file upload

Module 3: Database Queries, SQLAlchemy, and Dynamic Webpages

Introduction

In Module 2, we scraped and cleaned data from **Grad Café**. In Module 3, we are going to use that data to learn more about the graduate-school application entries submitted to the site.

This assignment will introduce you to querying relational databases using **SQL** and interacting with those databases using an **Object-Relational Mapper (ORM)**.

You will:

1. Load your cleaned Grad Café data into a PostgreSQL database.
2. Query the database using raw SQL.
3. Represent and query the same database using SQLAlchemy.
4. Create a dynamic Flask webpage that displays your analysis.
5. Connect your webpage to your Module 2 scraper so that new data can be added.
6. Think critically about the limitations of analyzing anonymous, self-submitted data.

Skills: SQL, Data Cleaning, PostgreSQL, `psycopg`, SQLAlchemy ORM, Flask, JSON, HTML, CSS



IT'S NOT EXACTLY WHAT I MEANT WHEN
I SAID I NEED AN SQL NINJA...



Proh@Dataedo

Assignment Overview

In this assignment you will:

1. Load collected data into a PostgreSQL database using `psycopg`.
2. Carry out data analysis using SQL queries to answer questions about Grad Café submissions.
3. Create a SQLAlchemy ORM representation of your database.
4. Repeat selected queries using SQLAlchemy instead of handwritten SQL.
5. Compare raw SQL and ORM-based database access.
6. Display your analysis results on a dynamic Flask webpage.
7. Allow users to pull newly available Grad Café data into the database.
8. Write two paragraphs about the limitations of using Grad Café and similar self-submission sites as data sources.

After completing this assignment, you will have a PostgreSQL relational database containing your scraped data, Python code for interacting with the database using both SQL and SQLAlchemy, and a dynamic Flask application that displays your results.

BEFORE YOU BEGIN: Pull all your code (and data) from your `module_2` folder into a new `module_3` folder.

Part 1: Load Data into a PostgreSQL Database

Create a new file called: `load_data.py`

This program should take the cleaned applicant data you produced in Module 2 and load it into a PostgreSQL database (you may reuse existing functionality / move around content from `clean.py` and `scrape.py`).

You may follow the PostgreSQL setup directions provided in lecture or use another PostgreSQL environment approved for the course.

Your Python program should connect to PostgreSQL using `psycopg` or `psycopg3`.

Important

Do **not** commit database passwords, API keys, `.env` files containing credentials, or other secrets to GitHub.

Database connection information should instead be stored using environment variables or another method that keeps credentials outside your repository.

Required Database Table

Your database must contain a single table called `applicants`.

Column Name	Data Type	Description
<code>p_id</code>	integer	Unique identifier
<code>program</code>	text	University and Department/Program
<code>comments</code>	text	Applicant comments
<code>date_added</code>	date	Date entry was added
<code>url</code>	text	Link to Grad Café entry
<code>status</code>	text	Admission status
<code>term</code>	text	Intended start term

Column Name	Data Type	Description
us_or_international	text	Applicant nationality classification
gpa	float	Applicant GPA
gre	float	GRE Quantitative score
gre_v	float	GRE Verbal score
gre_aw	float	GRE Analytical Writing score
degree	text	Degree type
llm_generated_program	text	LLM-generated department/program
llm_generated_university	text	LLM-generated university

Your loader should:

- handle missing values appropriately;
- avoid failing when optional values such as GPA, GRE scores, or comments are missing;
- insert the cleaned Module 2 data into PostgreSQL; and
- avoid creating unnecessary duplicate records when the loader is run more than once.

Example

Your database should look approximately like the following:

p_id	integer	program	text	comments	text
1		information studies, mcgill university		Ignore status. Did any of you apply for the MIST Fellowship for Black	
2		information, mcg		Ignore status. Did any of you apply for the MIST Fellowship for Black	
3		mathematics, university of british columbia			
4		chemistry, old dominion university		Accepted with GTA.	
5		environmental sciences, southern illinois university edwardsville		Accepted with partial funding	
6		physics, georgetown university		Has declined this offer. Good luck to those who are on the waitlist. S	
7		statistics, bowling green state university		Does anyone receive funding decisions? If you received it and you a	
8		philosophy, university of connecticut		Ignore status. Who has received rejection notices from UConn? I wa	
9		philosophy, university of connecticut		Ignore status. Who has received rejection notices from UConn? I wa	
10		atmospheric sciences, university of maryland			
11		no			

Your exact entries and number of rows will be different because Grad Café continues to receive submissions.

Part 2: SQL Query Analysis

Create a file called:

query_data.py

For this portion of the assignment, you must answer the questions below using **SQL queries**.

You may execute your queries through `psycopg`, but the analysis itself must be expressed in SQL.

Later in the assignment, you will repeat several of these analyses using SQLAlchemy.

Output Formatting Requirements

Please use the following formatting rules consistently in:

- your console output;
- your analysis PDF; and
- your Flask webpage.

Result	Required Format
Counts	Whole numbers
Percentages	2 decimal places and %
Average GPA	2 decimal places
Average GRE Quantitative	2 decimal places
Average GRE Verbal	2 decimal places
Average GRE Analytical Writing	2 decimal places

For example:

Fall 2026 applicant count: 19,290

Percent international: 50.09%

Average GPA: 3.79

Average GRE Quantitative: 164.88

Missing Values

When calculating an average, include only applicants who provide that **particular metric**.

For example:

- an applicant with a GPA but no GRE score should still contribute to the GPA average;
- an applicant with a GRE Verbal score but no GRE Analytical Writing score should still contribute to the GRE Verbal average.

Do **not** require an applicant to have all four values in order to contribute to an individual average.

Unless otherwise specified, your text matching should be reasonably robust to differences in capitalization.

Required SQL Questions

Question 1

How many entries in your database are from applicants who applied for Fall 2026?

Use the `term` field.

Report a whole-number count.

Example:

Fall 2026 applicant count: 19290

Question 2

Among entries that provide a nationality classification, what percentage are international students?

For this assignment:

- `American` does **not** count as international.
- `Other` does **not** count as international.
- Missing or blank values should be excluded from the denominator.

Therefore:

Numerator: entries classified as international

Denominator: all entries with a usable nationality classification

Report the result to **two decimal places**.

Example:

Percent international: 50.09%

Question 3

What are the average GPA, GRE Quantitative, GRE Verbal, and GRE Analytical Writing scores of applicants who provide each metric?

Calculate each average separately.

Report each value to **two decimal places**.

Example:

Average GPA: 3.79

Average GRE Quantitative: 164.88

Average GRE Verbal: 160.38

Average GRE Analytical Writing: 5.17

Question 4

What is the average GPA of American applicants who applied for Fall 2026?

Include only entries that:

- are for Fall 2026;
- are classified as **American**; and
- provide a GPA.

Report the result to **two decimal places**.

Question 5

What percentage of Fall 2025 entries are acceptances?

Use:

Numerator: Fall 2025 entries whose cleaned **status** indicates an acceptance

Denominator: all Fall 2025 entries

Report the result to **two decimal places**.

Example:

Fall 2025 acceptance percentage: 39.28%

Question 6

What is the average GPA of accepted applicants who applied for Fall 2026?

Include only entries that:

- are for Fall 2026;
- have a status indicating acceptance; and
- provide a GPA.

Report the result to **two decimal places**.

Question 7

How many entries are from applicants who applied to Johns Hopkins University for a master's degree in Computer Science?

For this question, use the **original downloaded fields**, particularly:

- **program**
- **degree**

Your matching should reasonably recognize:

- Johns Hopkins University / JHU;
- Computer Science; and
- master's degree entries.

Report a whole-number count.

Question 8

Using the **original downloaded fields**:

How many Fall 2026 entries are acceptances from applicants applying for a PhD in Computer Science at one of the following universities?

- Georgetown University
- Massachusetts Institute of Technology / MIT
- Stanford University
- Carnegie Mellon University

Your query should simultaneously restrict entries by:

- Fall 2026;
- acceptance status;
- PhD degree;
- Computer Science; and
- one of the four universities listed above.

Report a whole-number count.

Question 9

Repeat **Question 8**, but identify the university and program using:

- `llm_generated_program`
- `llm_generated_university`

instead of the original downloaded university/program information.

Continue using the appropriate original fields for:

- term;
- degree; and
- admission status.

Report:

1. the Question 8 count;
2. the Question 9 count; and
3. the difference between them.

For example:

Original-field count: 14

LLM-field count: 17

Difference: +3

In your written analysis, briefly explain why you think the results may or may not differ.

Part 3: Create Two Questions of Your Own

Come up with **two additional questions** that you are curious to answer using your database.

For each:

1. State the question in ordinary English.
2. Write a SQL query that answers the question.
3. Report your result.

Your questions should involve meaningful analysis of the database.

Good questions will generally use one or more of the following:

- filtering;
- aggregation;
- grouping;
- comparisons;
- averages;
- counts;
- percentages.

A question that simply retrieves arbitrary rows using `SELECT *` is not sufficient.

Part 4: SQL Analysis PDF

Create a file called:

`query_results.pdf`

Your PDF should include all **11 analysis questions**:

- Questions 1–9; and
- your two original questions.

For **each question**, provide:

1. The question in words.
2. Your final result.
3. The SQL query used.
4. A short explanation of what the query does and why it answers the question.

Your complete executable SQL-query code should also appear in:

`query_data.py`

Part 5: SQLAlchemy ORM

So far, you have interacted with PostgreSQL by writing SQL directly.

Another common way for Python programs to interact with relational databases is through an **Object-Relational Mapper**, or **ORM**.

An ORM maps database tables to Python classes and database rows to Python objects.

For this assignment, we will use **SQLAlchemy 2.x**.

Part A: Create an Applicant Model

Create:

`models.py`

Using SQLAlchemy, define a model called:

`Applicant`

The model should map to the **existing** `applicants` **table** in PostgreSQL.

Your `Applicant` model must contain all of the fields from the database schema above using appropriate SQLAlchemy types.

`p_id` should be the primary key.

You should also configure the SQLAlchemy:

- `Engine`; and
- `Session`

needed to connect your application to PostgreSQL.

Important

SQLAlchemy should interact with the **same PostgreSQL database and same** `applicants` **table** used by `load_data.py`.

Do **not** create a second copy of your applicant data specifically for SQLAlchemy.

Part 6: Query the Database Using SQLAlchemy

Create: `orm_queries.py`

Using SQLAlchemy rather than handwritten SQL, repeat the following analyses:

- Question 1
- Question 4
- Question 5
- Question 8
- Question 9
- **one of your two original questions**

Where the questions are equivalent, your SQLAlchemy answers should match your raw SQL answers.

You may find functionality such as the following useful:

`select()``where()``func.count()``func.avg()``and_()``or_()`

and SQLAlchemy `Session` objects.

Important

For this part of the assignment, you may **not bypass the ORM by submitting raw SQL**.

For example, do not solve this section using:

`text("SELECT ...")`

and do not use a raw `psycopg` cursor inside `orm_queries.py`.

The purpose of this section is to practice expressing database operations using SQLAlchemy.

Your ORM output should follow the **same decimal-place and formatting requirements** used for the raw SQL portion.

Part 7: Compare SQL and SQLAlchemy

In your `README.md`, select **one question** that you answered using both raw SQL and SQLAlchemy.

Include:

1. The raw SQL query.
2. The corresponding SQLAlchemy query.
3. A **3–5 sentence comparison** of the two approaches.

Your comparison should identify:

- at least one advantage of using an ORM; and
- at least one advantage of writing SQL directly.

You might consider:

- readability;
- abstraction;

- debugging;
- portability;
- flexibility; or
- control over the resulting query.

There is no requirement to argue that one approach is always superior.

Part 8: Flask Webpage

Part A: Analysis Page

In class, we learned how to connect Flask webpage frontends to PostgreSQL databases.

Now, you will put that experience into practice.

Create a **single dynamic Flask webpage** that displays the results of your analysis.

For this portion of the assignment, database reads used to populate your webpage should use your **SQLAlchemy** `Applicant` **model**.

You may reuse functions from `orm_queries.py` rather than duplicating identical database logic inside your Flask routes.

Your webpage should:

- retrieve analysis results dynamically from PostgreSQL;
- display the required analysis results;
- display your two original questions and answers;
- be organized and readable; and
- use CSS styling.

Your webpage does **not** need to look identical to the sample.

Analysis

How many entries do you have in your database who have applied for Fall 2024?

Answer: Applicant count: 19290

What percentage of entries are from International students (not American or Other) (to two decimal places)?

Answer: Percent International: 50.09

What is the average GPA, GRE, GRE V, GRE AW of applicants who provided these metrics?

Answer: Average GPA: 3.79, Average GRE: 164.88, Average GRE V: 160.38, Average GRE AW: 5.17

What is the average GPA of American students in Fall 2024?

Answer: Average GPA American: 3.78

What percent of entries for Fall 2024 are Acceptances (to two decimal places)?

Answer: Acceptance percent: 39.28

What is the average GPA of applicants who applied for Fall 2024 who applied for Acceptances?

Answer: Average GPA Acceptance: 3.79

Part 9: Pull Data

Add a button to your Analysis webpage called:

Pull Data

When a user clicks **Pull Data**, your application should call your Module 2 scraping code and retrieve newly available Grad Café entries.

New usable records should then be added to your PostgreSQL database.

Your webpage should include a short, user-friendly explanation of what this button does.

For example:

Pull Data checks Grad Café for newly submitted application results and adds new records to the database. Depending on the amount of new data available, this may take some time.

Your application should **not start another scraping process if one is already running**.

Using a Python subprocess is one reasonable way to implement this behavior.

Part 10: Update Analysis

Add a second button near the top-right of your Analysis webpage called:

Update Analysis

When clicked, this button should:

1. re-query your PostgreSQL database; and
2. display the most up-to-date analysis results currently available.

The **Update Analysis** button should **not initiate another scrape**.

If a **Pull Data** request is currently running:

- Update Analysis should not interfere with that process; and
- the webpage should tell the user that new data is currently being retrieved.

If no Pull Data request is running, the page should refresh using the most current data in PostgreSQL.

Please provide useful user-facing status messages rather than silently ignoring requests.

Part 11: Written Reflection

Create:

[limitations.pdf](#)

Write **two substantive paragraphs** about the limitations of performing statistical analysis over anonymously submitted, self-reported data such as Grad Café.

Your response should discuss issues such as:

- selection bias;
- self-reporting bias;
- missing values;
- incorrect or inconsistent information;
- representativeness;
- who chooses to submit results;
- who may be absent from the dataset; and
- whether applicants with particularly positive, negative, or unusual outcomes may be more likely to post.

You should connect your discussion to at least **one result from your own analysis**.

For example, suppose a broader reference population has an average GRE Quantitative score around 157, while your Grad Café dataset has an average near 165.

Why might that happen?

Does it mean that the broader applicant population actually has an average score of 165?

Might people with particularly high GRE scores be more likely to include them?

Might certain programs, universities, or applicant populations be disproportionately represented on Grad Café?

Your goal is to distinguish between:

what your database mathematically contains

and

what conclusions the database actually allows you to make about the real world.

Submission Instructions

Canvas Submission

Your entire Module 3 submission should be submitted to Canvas as **one ZIP file**.

Name your ZIP file:

`module_3.zip`

Do **not** upload each file separately.

Your ZIP file should contain your completed `module_3` folder and all files necessary to evaluate and run your project.

Required Deliverables

Your `module_3.zip` should contain:

1. `load_data.py`
2. `query_data.py`
3. `models.py`
4. `orm_queries.py`
5. `query_results.pdf`
6. `limitations.pdf`

7. Your Flask application code
8. Your HTML template(s)
9. Your CSS/static files
10. Any Module 2 scraping code required for Pull Data functionality
11. `README.md`
12. `requirements.txt`
13. Screenshots showing:
 - raw SQL console output;
 - SQLAlchemy ORM console output; and
 - the running Flask webpage.
14. `github.txt`

Your `github.txt` file should contain the **SSH URL to your private GitHub repository**.

For example:

```
git@github.com:username/repository-name.git
```

Please make sure the GitHub repository contains the same final version of your work that appears in your submitted ZIP file.

Do not include:

- passwords;
- API keys;
- `.env` files containing credentials;
- virtual environments;
- `__pycache__`;
- database dumps containing secrets; or
- other unnecessary generated files.

Please remember to both:

1. commit your final work to your private GitHub repository; and
2. submit `module_3.zip` to Canvas.

Sample Query Output

Your console output may look approximately like the following:

```
~/M3-HW$ python query_data.py
Applicant count: 19290
International count: 9933
US count: 9262
Other count: 635
Percent International 50.09
Average GPA: 3.7872697974217457, Average GRE: 164.87784679089026,
Average GRE V: 160.37602927721866, Average GRE AW: 5.166470881035339
Average GPA American: 3.778753731343264
Acceptance count: 7578
Acceptance percent: 39.28
Average GPA Acceptance: 3.7940969072164603
JHU Masters Computer Science count: 11
~/M3-HW$
```

The screenshot above is from an older version of the assignment.

For this assignment, please format numeric results according to the requirements above.

For example, instead of displaying:

Average GPA: 3.7872697974217457

display:

Average GPA: 3.79

Instead of:

Average GRE: 164.87784679089026

display:

Average GRE Quantitative: 164.88

Grading Rubric

Total: 100 points

Category	Points	Criteria
GitHub Repository Setup and Submission	5	<code>github.txt</code> contains the correct private GitHub SSH URL: 2 pts. Repository contains a clearly organized <code>module_3</code> folder with required files: 2 pts. Canvas ZIP and final GitHub version correspond: 1 pt.
PostgreSQL Database Setup and Data Loading	18	<code>load_data.py</code> successfully connects to PostgreSQL using <code>psycopg</code> : 3 pts. Required <code>applicants</code> table and schema are implemented with appropriate types and primary key: 5 pts. Cleaned Module 2 data is successfully

Category	Points	Criteria
		inserted: 4 pts. Missing values, formatting, and repeated loading are handled without major corruption or unnecessary duplication: 3 pts. Code is readable, includes reasonable error handling, and credentials are not committed: 3 pts.
Raw SQL Query Analysis	22	<code>query_data.py</code> correctly answers Questions 1–9 using SQL: 9 pts. Queries apply required term, status, nationality, university, program, and degree filtering correctly: 4 pts. Percentages, denominators, NULL handling, averages, and required decimal formatting are correct: 3 pts. Two meaningful original questions are formulated and answered: 3 pts. <code>query_results.pdf</code> includes results, queries, and explanations: 3 pts.
SQLAlchemy ORM	15	<code>models.py</code> contains a correct <code>Applicant</code> model mapping the existing table: 5 pts. SQLAlchemy Engine and Session are configured using modern conventions: 2 pts. <code>orm_queries.py</code> correctly repeats Questions 1, 4, 5, 8, 9, and one original question: 4 pts. ORM work does not bypass SQLAlchemy using handwritten SQL/cursors: 1 pt. Flask database reads use the ORM: 2 pts. README contains a meaningful SQL-versus-ORM comparison: 1 pt.
Flask Webpage Functionality	20	Flask application connects to PostgreSQL through SQLAlchemy: 3 pts. Required analysis results are retrieved dynamically and displayed correctly: 5 pts. Page is organized, readable, and meaningfully styled using CSS: 3 pts. Pull Data button is functional and explained to users: 3 pts. Update Analysis refreshes results and appropriately handles an active scrape: 4 pts. Application runs without major runtime errors: 2 pts.
Scraping and Data Refresh Integration	8	Module 2 scraping functionality is appropriately reused/integrated: 2 pts. New records are added without overwriting or corrupting existing usable data: 2 pts. Conflicting simultaneous operations are prevented: 2 pts. Clear user-facing status/error messages are provided: 2 pts.
Written Reflection: Data Limitations	7	<code>limitations.pdf</code> contains two substantive paragraphs about limitations of anonymous/self-submitted data: 3 pts. Reflection meaningfully addresses selection bias, missingness, reliability, representativeness, or related issues: 2 pts. Reflection connects the limitations to at least one analysis result or appropriate comparison: 1 pt. Writing is clear and proofread: 1 pt.
Documentation, Screenshots, and Requirements	5	<code>README.md</code> provides sufficient setup/run instructions for the database, SQL scripts, ORM, scraper, and Flask application: 2 pts. <code>requirements.txt</code> includes necessary dependencies: 1 pt. Screenshots demonstrate raw SQL output, ORM output, and running Flask webpage: 2 pts.
TOTAL	100	

